Module 11: Events and Logging

Real-Time Monitoring & Event Streaming

Event-Driven Architecture

Event System Overview

- **Real-Time Monitoring**: Live blockchain event streaming
- **WebSocket Integration**: Persistent client connections
- **Subscription Management**: Flexible event filtering and delivery
- **Comprehensive Logging**: Complete audit trail of all operations

Key Capabilities

- **Blockchain Monitoring**: Program logs, account changes, slot updates
- **Event Filtering**: Advanced filtering and subscription rules
- **Real-Time Delivery**: WebSocket and webhook notifications
- **Historical Replay**: Event replay for state reconstruction

Architecture Overview

Event Types & Monitoring

Supported Event Types

```
enum EventType {
  TRANSACTION_CONFIRMED = 'TRANSACTION_CONFIRMED',
  ACCOUNT_CHANGED = 'ACCOUNT_CHANGED',
  PROGRAM_LOG = 'PROGRAM_LOG',
  CPI_INVOCATION = 'CPI_INVOCATION',
  BLOCK_PRODUCED = 'BLOCK_PRODUCED',
  SLOT_UPDATED = 'SLOT_UPDATED'
}
```

Blockchain Monitoring Setup

```
async startBlockchainMonitoring(): Promise<void> {
  // Program log monitoring
  this.connection.onLogs('all', (logs) => {
    this.handleProgramLogs(logs);
  });
```

Event Processing Pipeline

Event Flow Architecture

1. 🔍 Event Detection → Blockchain Monitoring
2. 📝 Event Creation → createEvent()
3. 💾 Database Storage → Persistent Storage
4. 📡 WebSocket Emission → Real-time Broadcasting
5. 🎯 Subscription Filtering → Targeted Delivery
6. 🌐 Webhook Delivery → HTTP Callbacks

Event Entity Structure

```
@Entity()
export class Event {
  @PrimaryGeneratedColumn('uuid')
  id: string;

  @Column({
    type: 'enum',
    enum: EventType
  })
  eventType: EventType;
  @Column()
  source: string;  // Account/Program ID
```

Subscription Management

WebSocket Gateway Implementation

```
@WebSocketGateway({
  namespace: '/events',
  cors: true
})
export class EventsGateway {
  @WebSocketServer()
  server: Server;

  private clients = new Map<string, Socket>();

  handleConnection(client: Socket) {
    this.clients.set(client.id, client);
    client.join(`client:${client.id}`);
  }

  handleDisconnect(client: Socket) {
    this.clients.delete(client.id);
  }

  @SubscribeMessage('subscribe')
  handleSubscribe(client: Socket, data: SubscribeDto) {
    // Create subscription
    const subscription = await this.subscriptionService.createSubscription({
      clientId: client.id,
      eventTypes: data.eventTypes,
      filters: data.filters
    });

    // Join subscription room
    client.join(`subscription:${subscription.id}`);
  }
  emitEvent(event: Event) {
    // Broadcast to all subscribers
```

Advanced Event Filtering

Filter System Architecture

```
interface EventFilter {
  id: string;
  name: string;
  eventType: EventType;
  conditions: FilterCondition[];
  action: 'include' | 'exclude' | 'transform';
  isActive: boolean;
}

interface FilterCondition {
  field: string;          // Event field to check
  operator: 'eq' | 'ne' | 'gt' | 'lt' | 'contains' | 'regex';
  value: any;             // Comparison value
}
```

Filter Application

Real-Time Features

WebSocket Real-Time Streaming

- **Persistent Connections**: Long-lived client connections
- **Room-Based Messaging**: Targeted event broadcasting
- **Connection Recovery**: Automatic reconnection handling
- **Load Balancing**: Scalable WebSocket server architecture
- **Message Buffering**: Event queuing during network issues

Real-Time Monitoring Capabilities

```
// Account monitoring subscription
@Post('/monitor/account/:accountId')
async subscribeToAccount(
  @Param('accountId') accountId: string,
  @Body() subscription: AccountSubscriptionDto
): Promise<SubscriptionResult> {
  // Start monitoring account changes
```

Event Replay & Historical Analysis

Event Replay Implementation

```
async getEventsForReplay(replayDto: EventReplayDto): Promise<Event[]> {
  const query = this.eventRepository.createQueryBuilder('event');

  // Apply time range filter
  if (replayDto.startTime) {
    query.andWhere('event.createdAt >= :startTime', {
      startTime: replayDto.startTime
    });
  }

  if (replayDto.endTime) {
    query.andWhere('event.createdAt <= :endTime', {
      endTime: replayDto.endTime
    });
  }

  // Apply event type filter
  if (replayDto.eventTypes?.length) {
    query.andWhere('event.eventType IN (:...eventTypes)', {
      eventTypes: replayDto.eventTypes
    });
  }

  // Apply source filter
  if (replayDto.sources?.length) {
    query.andWhere('event.source IN (:...sources)', {
      sources: replayDto.sources
    });
  }
```

API Endpoints

Event Management

- `POST /events` - Create custom event
- `GET /events` - Query events with filtering
- `PUT /events/:id` - Update event status
- `GET /events/replay` - Get events for replay

Subscription Management

- `POST /events/subscriptions` - Create event subscription
- `GET /events/subscriptions/:clientId` - Get client subscriptions
- `PUT /events/subscriptions/:id` - Update subscription
- `DELETE /events/subscriptions/:id` - Remove subscription

Key Takeaways

Event System Benefits

- **Real-Time Monitoring**: Live blockchain event streaming
- **Flexible Subscriptions**: WebSocket and webhook delivery options
- **Advanced Filtering**: Granular event selection and processing
- **Historical Analysis**: Complete event replay and statistics

SVM Event Advantages

- **High-Frequency Events**: Handle thousands of events per second
- **Parallel Processing**: Multiple event streams processed simultaneously
- **State Synchronization**: Real-time state updates across distributed systems
- **Audit Trail**: Immutable event history for compliance and debugging